

NOI.PH-YTP Junior Track: C++ Programming 2

Control structures, dynamic types, compound types

Aldrich Ellis Asuncion

Contents

1	References, pointers, and dynamic types	2
1.1	References	2
1.1.1	The basic idea	2
1.1.2	Declaring references	3
1.2	Pointers	4
1.2.1	The basic idea	4
1.2.2	The dereference operator	5
1.3	Dynamic types: <code>std::vector</code> and <code>std::string</code>	7
1.3.1	The basic idea behind how <code>std::vector</code> works	7
1.3.2	Using <code>std::vector</code>	11
1.3.3	Using <code>std::string</code>	14
2	Control structures	17
2.1	The basics you should know	17
2.2	Some common patterns	18
2.2.1	Reading a list of n integers	18
2.2.2	Reading an $R \times C$ grid	19
2.2.3	Passing vectors to functions	20
3	Pairs, tuples, and structs	22
3.1	<code>std::pair</code>	22
3.2	<code>std::tuple</code>	25
3.3	Structs	26
3.4	Custom sorting	26
3.5	A couple notes on list-initialization	26

1 References, pointers, and dynamic types

Remark 1.1. What follows is a simplified version of [learncpp.com – lvalue references](#) and [learncpp.com – introduction to pointers](#).

This handout follows a simplified presentation of references which is sufficient for competitive programming, but is not a fully accurate description of how C++ works. Don't worry, it is normal to learn C++ by slowly refining your understanding of the language. It's near impossible to learn C++ by seeing all the details all at once on your first go.

But for the sake of accuracy, I'll be adding footnotes. **You can ignore all footnotes.**

1.1 References

1.1.1 The basic idea

In C++, a **reference**¹ is an alias for an existing object. Once a reference has been defined, any operation on the reference is applied to the object being referenced. To denote a reference type, we use an ampersand (&) in the type declaration:

```
1 int      // a normal int type
2 int&     // an reference to an int object
3 double&  // an reference to a double object
```

```
1 #include <iostream>
2 int main() {
3     int x { 5 };    // x is a normal integer variable
4     int& y { x };   // y is an alias for variable x
5     std::cout << x << " " << y << std::endl; // output: 5 5
6     x = 6;
7     std::cout << x << " " << y << std::endl; // output: 6 6
8     y = 7;
9     std::cout << x << " " << y << std::endl; // output: 7 7
10 }
```

Notice that `x` and `y` are now essentially two “variable names” which refer to the same variable. Compare what happened above to if the second variable wasn't a reference.

```
1 #include <iostream>
2 int main() {
3     int x { 5 };
4     int y { x }; // y is a normal integer variable, note the lack of &
5     std::cout << x << " " << y << std::endl; // output: 5 5
6     x = 6;
7     std::cout << x << " " << y << std::endl; // output: 6 5
8     y = 7;
9     std::cout << x << " " << y << std::endl; // output: 6 7
10 }
```

¹Actually, an lvalue reference (pronounced ell-value)

1.1.2 Declaring references

When a reference is initialized with an object, we say it is **bound** to that object. The process by which such a reference is bound is called **reference binding**. The object (or function) being referenced is sometimes called the **referent**.

```
1 int x { 5 };
2 int& y { x };
```

Given the above code, we can say the following:

- y is bound to x.
- y is a reference to x.
- x is the referent of y.

Perhaps surprisingly, references are not objects in C++. A reference is not required to exist or occupy storage. If possible, the compiler will optimize references away by replacing all occurrences of a reference with the referent. However, this isn't always possible, and in such cases, references may require storage. Your mental picture of what it means when we say “y is a reference to x” can be something like this:

addresses	memory	
0x7fffb04da37c	57 68 79 20	} x ← y
0x7fffb04da380	61 72 65 20	
0x7fffb04da384	79 6F 75 20	
0x7fffb04da388	72 65 61 64	
0x7fffb04da38c	69 6E 67 20	
0x7fffb04da390	74 68 69 73	
0x7fffb04da394	3F 20 47 61	
0x7fffb04da398	6C 69 6E 67	
0x7fffb04da39c	20 6D 6F 20	
0x7fffb04da3A0	6E 61 6D 61	
0x7fffb04da3A4	6E 2E 2E 2E	

When you declare a reference, it *must* be initialized to an alias for an existing modifiable variable². In most cases³, the type of the reference must match the type of the referent. Once initialized, a reference in C++ cannot be *reseated*, meaning it cannot be changed to reference another object.

```
1 const int x {6};
2 int& a;      // error: reference not initialized
3 int& b {5};  // error: 5 is not a variable
4 int& c {x};  // error: x is constant
5
6 double y {3.14};
7 int z {7};
8 int& d { y }; // error: reference to int cannot bind to double variable
9 double& e { z }; // error: reference to double cannot bind to int variable
```

That's all we'll mention about references for now. They might not seem that useful, but are crucial for understanding some later concepts in this handout.

²Actually, the rule is “Lvalue references must be bound to a modifiable lvalue.” `int& b{5}` fails since 5 is an rvalue, while `int& c {x}`; fails since x is `const`.

³Like pointers, the type of the reference can be a superclass of the type of the referent.

1.2 Pointers

1.2.1 The basic idea

Remember this picture?

addresses	memory
0x7fffb04da37c	57 68 79 20
0x7fffb04da380	61 72 65 20 } x
0x7fffb04da384	79 6F 75 20
0x7fffb04da388	72 65 61 64
0x7fffb04da38c	69 6E 67 20
0x7fffb04da390	74 68 69 73
0x7fffb04da394	3F 20 47 61
0x7fffb04da398	6C 69 6E 67
0x7fffb04da39c	20 6D 6F 20
0x7fffb04da3A0	6E 61 6D 61
0x7fffb04da3A4	6E 2E 2E 2E

When we are working with a variable like `x`, we don't usually care about the *memory address* that `x` is located at. But we do have access to this information.

The **address-of operator** (`&`) returns the memory address of its operand. In other words, `&x` obtains the address of the variable `x`. Try the following code, and you'll get a memory address.

```
1 int x{ 5 };
2 std::cout << x << '\n'; // print the value of variable x
3 std::cout << &x << '\n'; // print the memory address of variable x
```

A **pointer** is a variable which holds a memory address. To denote a pointer type, we use an asterisk (`*`) in the type declaration:

```
1 int // a normal int type
2 int* // an pointer to an int object (a memory address to an int)
3 double* // an pointer to a double object (a memory address to a double)
```

Here is an example.

```
1 int x {5};
2 int* p {&x};
```

Notice in the picture⁴ on the right that `p` is like a regular variable. It just holds the memory address of `x`.

In this case, we say `p` **points** to `x`.

addresses	memory
0x7fffb04da37c	57 68 79 20
0x7fffb04da380	00 00 00 05 } x
0x7fffb04da384	00 00 7F FF } p
0x7fffb04da388	B0 4D A3 80
0x7fffb04da38c	69 6E 67 20
0x7fffb04da390	74 68 69 73
0x7fffb04da394	3F 20 47 61
0x7fffb04da398	6C 69 6E 67
0x7fffb04da39c	20 6D 6F 20
0x7fffb04da3A0	6E 61 6D 61
0x7fffb04da3A4	6E 2E 2E 2E

⁴Picture assumes `int` is 32 bits and pointers are 64 bits. Pointers are basically **unsigned long long**. Also, the values in the memory view are technically wrong, since the picture supposes big-endian, but the standard used by systems now is little-endian.

1.2.2 The dereference operator

Getting the address of a variable isn't very useful by itself.

The most useful thing we can do with an address is access the value stored at that address. The **dereference operator** (*) (also occasionally called the **indirection operator**) returns the (modifiable) variable⁵ at a given memory address.

Try running the following code and following along with what is happening.

```
1  #include <iostream>
2  int main() {
3      int x {5};
4      int y {7};
5
6      // p points to x
7      int* p {&x};
8
9      // print x directly
10     std::cout << x << std::endl;
11
12     // print the address stored in p
13     std::cout << p << std::endl;
14
15     // print the variable at the address stored in p
16     std::cout << *p << std::endl;
17
18     // assign 2 to the variable at the address stored in p
19     *p = 2;
20     std::cout << x << std::endl;
21
22     // p now points to y
23     p = &y;
24
25     *p = 3;
26     std::cout << x << std::endl;
27     std::cout << y << std::endl;
28 }
```

Roughly speaking, here's how to understand the new syntax we've seen:

- &x means the location of x.
- *x means the variable pointed to by x.

Notice that pointers are more flexible than references, since we can *change what variable a pointer points to*. (line 23) Additionally, unlike references, pointers do not have to be initialized. Pointers do not actually have to point to a variable. A pointer that does not point to a variable is called a **null pointer**. We can assign the value **nullptr** to a pointer to indicate that it doesn't point to anything.

```
1  int* p {nullptr};
```

⁵The dereference operator returns an lvalue.

Remark 1.2. A single ampersand symbol `&` can mean one of three things in C++:

- When following a type name, `&` denotes a reference: `int& ref`.
- When used before a single value, `&` obtains an address: `&x`.
- When used between two values, `&` is the [bitwise AND operator](#), discussed later in this handout.

Remark 1.3. `nullptr` is a value, just like `true` and `false`. It was introduced in C++11. Before C++11, values such as `0` and `NULL` were used to denote null pointers. These are not recommended for use now.

For this handout, we will not be discussing pointers further, since a lot of the complicated work involving pointers will be handled behind the scenes. Understanding what a pointer is and what the `&` and `*` operations do to a pointer is enough for now.

Remark 1.4. I am leaving out a bunch of introductory C++ stuff, like `new` and `delete`, because they aren't super important to learn early on when you're learning C++ for competitive programming.

If you're looking to learn C++ to develop apps, games, etc. or for a job, you'll want to actually go through the *entirety* of learncpp.com.

1.3 Dynamic types: `std::vector` and `std::string`

1.3.1 The basic idea behind how `std::vector` works

A `std::vector`, defined in the `<vector>` header, is basically a resizable array. Like an array, all the elements of a vector have to be the same type.

```
1 // a vector containing 4 integers
2 std::vector<int> v {8, 4, 5, 9};
3
4 // a vector containing 3 doubles
5 std::vector<double> v {8.0, 4.2, 5.1};
```

We can use the `push_back` function to add elements to the end of a vector, and the `pop_back` function to remove elements.

```
1 #include <iostream>
2 #include <vector>
3
4 int main()
5 {
6     std::vector<int> v {8, 4, 5, 9};
7     // v is now [8, 4, 5, 9]
8
9     v.push_back(6);
10    v.push_back(2);
11
12    // Add two more integers to vector
13    v.pop_back();
14    // v is now [8, 4, 5, 9, 6]
15 }
```

Normally, when you declare variables and C-style arrays, you basically cannot change their size or move where they are in memory anymore. So how does a `std::vector` work? **What black magic does a vector do?**

Actually, behind the scenes, a `std::vector` creates a C-style array in memory as the program is running. Since it is not known in advance exactly how big the `std::vector` should be before the program is run, we say that the memory is **dynamically allocated**. Dynamically allocated memory goes on the heap segment of your program memory. Remember this diagram?



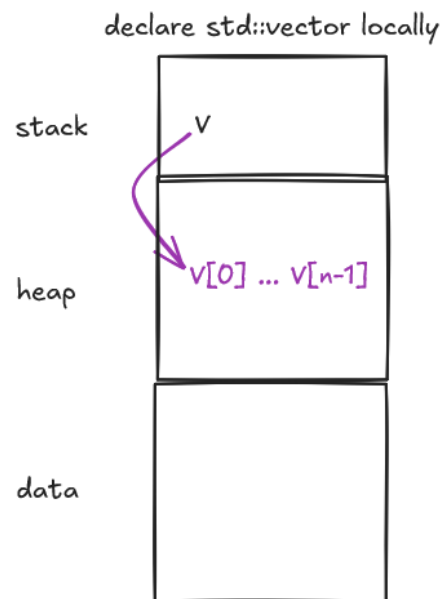
Consider the following code.

```
1  #include <iostream>
2  #include <vector>
3
4  int main()
5  {
6      std::vector<int> v {8, 4, 5, 9};
7  }
```

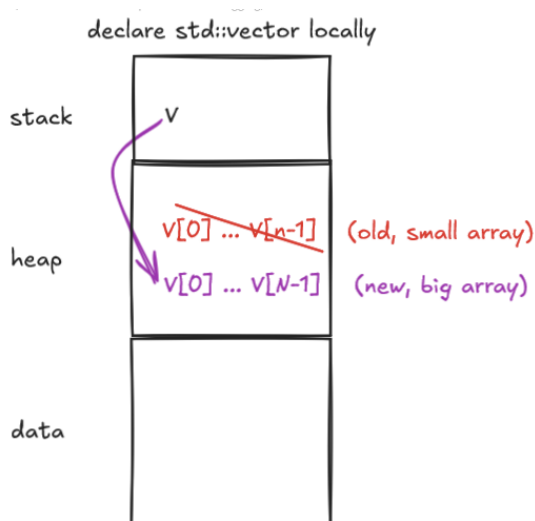
The variable `v` itself is on the stack, since it's declared inside a function. When the vector is created, it creates an array on the heap. The variable `v` just has a pointer to the array on the heap.

The array that is managed by `v` has *fixed-size*, just like a C-style array. Nothing magical there.

You don't have direct access to the array that is being managed by `v`. Even if we think we have 4 elements, it's possible that the array managed by the vector is actually, say, 8 elements long. This gives the vector "room to grow."



Most of the time, when you "add" elements to a vector, you're just making more use of the internal array that the vector manages. But what happens when we add so many elements that the array *runs out of space*? Well, the vector just does the following:



1. Make a new *larger* array on the heap.
2. Copy all the contents of the old array into the new array.
3. Make the vector point to the new array.
4. Discard the old array.

(Friendship ended with small array, now big array is my friend.)

So, a `std::vector` **doesn't actually "grow" like how you think it might**. It just manipulates memory behind the scenes to give the illusion that it can grow and shrink. By the way, this is basically also how lists in Python work.

One practical application of this knowledge is as follows. Remember these two programs?

```

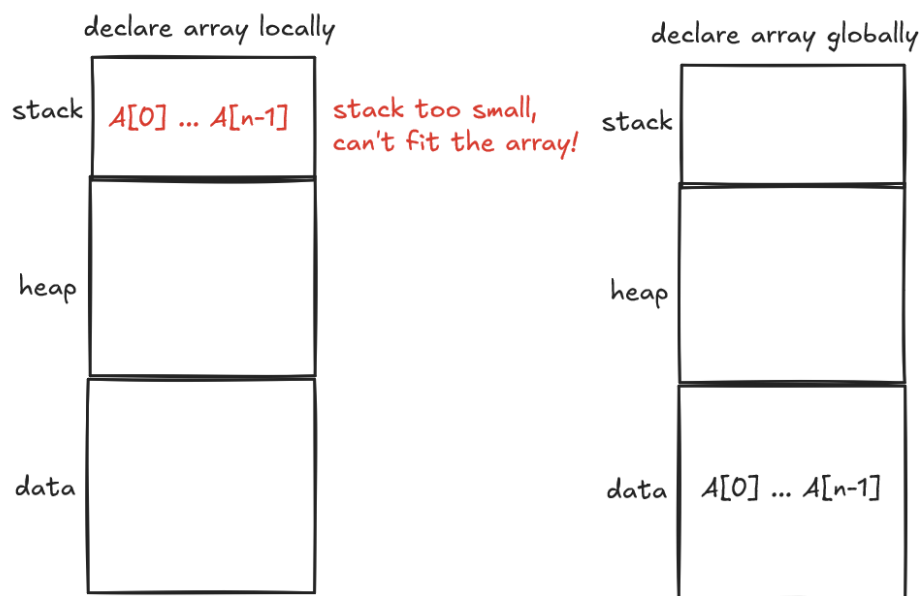
1  #include <iostream>
2  int main() {
3      int A[100000000] {};
4      std::cout << A[0] << std::endl;
5  }

1  #include <iostream>
2  int A[100000000] {};
3  int main() {
4      std::cout << A[0] << std::endl;
5  }

```

The first one crashes because local variables are placed on the stack, which has a smaller capacity compared to the heap and data segments.

The second program does not crash because global variables end up on the data segment, which has a large capacity.



To declare a vector with n value-initialized elements, we do something like the following. Note the use of parentheses instead of braces.

```

1  // declare a vector containing 1000 integers, all set to 0
2  std::vector<int> V(1000);

```

Now, which of the following programs do you think will crash?

```

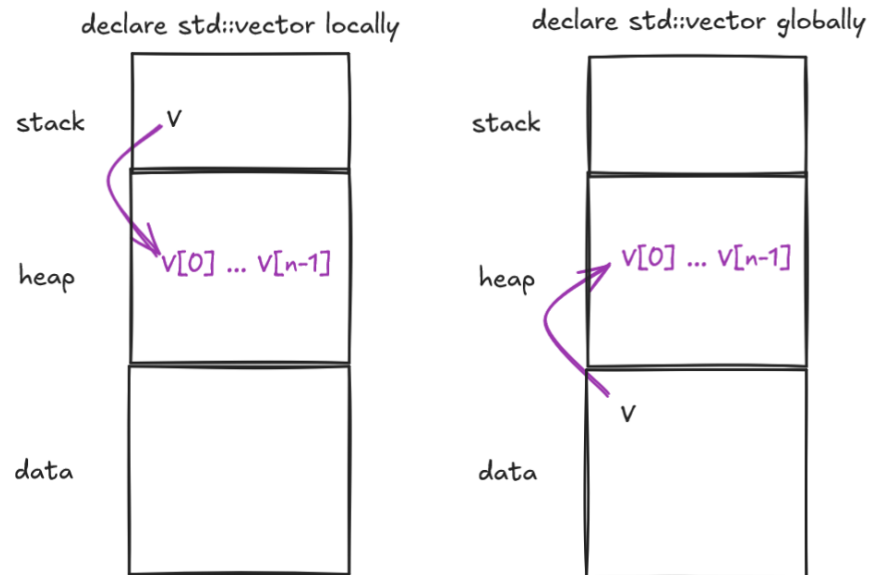
1  #include <iostream>
2  #include <vector>
3
4  std::vector<int> V(100000000);
5
6  int main() {
7      cout << V[0] << endl;
8  }

1  #include <iostream>
2  #include <vector>
3
4
5  int main() {
6      std::vector<int> V(100000000);
7      cout << V[0] << endl;
8  }

```

It turns out, neither of the two programs using vectors crash!

While the vector object would be on the stack or data segment, the dynamically-allocated array that it points to lives on the heap. And in either case, the heap is large enough to carry a large number of elements.



1.3.2 Using `std::vector`

Because a `std::vector` is effectively just a C-style array that is extended on demand, it has the following characteristics.

1. A `std::vector` is a homogenous container, meaning all of its elements must have the same type.
2. A `std::vector` allows fast access to individual elements, just like a C-style array.
3. Elements can only be added to and removed from the end of a `std::vector`, not the beginning of the vector.

Here, we lay out all the general syntax you need to know to use `std::vector` for competitive programming. Full details are of course available in [the documentation](#).

Initialization. We can use braces to initialize the vector with existing elements.

```
1  std::vector<int> primes { 2, 3, 5, 7, 11 };
2
3  // an empty vector
4  std::vector<int> V {};
```

We can use parentheses to initialize a vector with a certain number of elements.

```
1  // this vector contains one integer, which is 10
2  std::vector<int> V1 {10};
3
4  // this vector contains ten integers, all 0
5  std::vector<int> V2 (10);
```

Size queries. A `std::vector` has functions `size()` (returns number of elements) and `empty()` (checks if container has no elements).

```
1  std::vector<int> primes { 2, 3, 5, 7, 11 };
2  std::vector<int> V {};
```

```
3
4  std::cout << primes.size() << std::endl; // output: 5
5  std::cout << V.size() << std::endl; // output: 0
6
7  // The empty() function returns a bool.
8  // In C++, when a bool is printed using std::cout, true displays as 1 and false
   ↳ displays as 0.
9  // (Unlike Python, which prints the words "True" and "False"
10 std::cout << primes.empty() << std::endl; // output: 0 (false)
11 std::cout << V.empty() << std::endl; // output: 1 (true)
```

Printing out a vector. We can use the following *range for loop* to print out the elements of a vector. (Range for loops will be discussed in part 2 of this handout.)

```
1 std::vector<int> v {2, 3, 5, 7, 11};
2
3 for (auto& x : v)
4 {
5     std::cout << x << std::endl;
6 }
```

Accessing elements. The `[]` operator⁶ can be used to access elements of a vector.

```
1 std::vector<int> primes {2, 3, 5, 7, 11};
2 std::cout << primes[3] << std::endl; // output: 7
```

The `front()` and `back()` functions can also be used to directly access the first and last elements of a vector. In some cases, writing `back()` can denote the intent of your code clearer than say, `V[V.size() - 1]`.

```
1 std::vector<int> primes {2, 3, 5, 7, 11};
2 std::cout << primes.front() << std::endl; // output: 2
3 std::cout << primes.back() << std::endl; // output: 11
```

Assigning elements. The `[]` operator⁷ returns a *reference* to an element. So you can assign to it and it will modify the element of the vector.

```
1 std::vector<int> primes {2, 3, 5, 7, 11};
2 primes[2] = 42; // primes: {2, 3, 42, 7, 11}
```

The `front()` and `back()` functions *also* return references to the first and last elements respectively. So you can assign to them.

```
1 std::vector<int> primes {2, 3, 5, 7, 11};
2 primes.front() = 0;
3 primes.back() = -1;
4 // primes: {0, 3, 5, 7, -1}
```

Appending. The `push_back()` function adds an element to the end of the vector. The `pop_back()` function removes an element from the end of a vector.

```
1 std::vector<int> primes {2, 3, 5, 7, 11};
2 primes.push_back(12); // primes: {2, 3, 5, 7, 11, 12}
3 primes.pop_back();    // primes: {2, 3, 5, 7, 11}
4 primes.push_back(13); // primes: {2, 3, 5, 7, 11, 13}
```

⁶There's also an `at()` function, which does error checking, and has a `const` overload. But you probably won't need that for competitive programming. You shouldn't be writing code which results in an out-of-bounds vector access. And you'll definitely not be specifying functions as `const` in competitive programming.

⁷The `at()` function also returns a reference, so you can write `v.at(2) = 3`; for example.

Resetting contents. The `clear()` function removes all elements from a vector.

```
1 std::vector<int> V {2, 1, 3, 4, 0};
2 V.clear(); // V is now empty
```

The `assign()` function replaces the contents of a vector with copies of a given value.

```
1 std::vector<int> V {2, 1, 3, 4, 0};
2 // set V to 10 elements, each -1.
3 V.assign(10, -1);
4 // V: {-1, -1, -1, -1, -1, -1, -1, -1, -1, -1}
```

Unlike C-style arrays, you can assign a braced list to a vector using the `=` operator and it will replace the contents, even if its not during initialization of the vector.

```
1 std::vector<int> V {2, 1, 3, 4, 0};
2 V = {1,2,3,4,5,6};
```

Using `<algorithm>`. Similar to C-style arrays, we can use the `std::begin()` and `std::end()` functions from the `<iterator>` header.

```
1 std::vector<int> V {2, 1, 3, 4, 0};
2 std::sort(std::begin(V), std::end(V));
3 // V: {0, 1, 2, 3, 4}
```

However, a more natural syntax for C++ containers like `std::vector` is to use the `begin()` and `end()` functions of the vector itself.

```
1 std::vector<int> V {2, 1, 3, 4, 0};
2 std::sort(V.begin(), V.end());
3 // V: {0, 1, 2, 3, 4}
```

Comparing two vectors. The `==` and `!=` operators can be used to check if two vectors are equal (`==`) or not equal (`!=`). Two vectors are equal if they have the same number of elements and their corresponding elements are equal.

```
1 std::vector<int> a{1, 2, 3};
2 std::vector<int> b{1, 2, 3};
3 std::vector<int> c{7, 8, 9, 10};
4 std::vector<int> d{7, 9, 8, 10};
5
6 std::cout << (a == b) << std::endl; // output: 1 (true)
7 std::cout << (a != c) << std::endl; // output: 1 (true)
8 std::cout << (c == d) << std::endl; // output: 0 (false)
```

1.3.3 Using `std::string`

Those of you who already know C++ might be wondering why I'm only mentioning strings now. Well, `std::string` is basically an [optimized and specialized](#)⁸ version of `std::vector<char>`, meant to represent and work with character arrays.

`std::string` is defined in the `<string>` header (which is often already included by `<iostream>`, but is not guaranteed to be included by `<iostream>`).

In fact, *all of the functions described in the previous section also work with strings*. This is unlike some other languages where strings are “immutable”⁹. Here's another list of common functions. Full details are of course available in [the documentation](#).

Initialization and size.

```
1 // initializing an empty string
2 std::string s1;
3 // initializing a string
4 std::string s2 {"hello"};
5 // initializing a string using a list of characters
6 // yes you can do this i guess but why not just assign to "abc"
7 std::string s3 {'a', 'b', 'c'};
8 // strings can be printed out directly without a loop
9 std::cout << s1 << std::endl;
10 std::cout << s2 << std::endl;
11 std::cout << s3 << std::endl;
12 std::cout << s1.empty() << std::endl; // output: 1 (true)
13 std::cout << s2.size() << std::endl; // output: 5
```

Accessing and assigning elements.

```
1 std::string s {"boat"};
2 std::cout << s[1] << std::endl; // output: o
3 std::cout << s.front() << std::endl; // output: b
4 std::cout << s.back() << std::endl; // output: t
5 s[2] = 'o';
6 std::cout << s << std::endl; // output: boot
7 s.back() = 'r';
8 std::cout << s << std::endl; // output: boor
9 s.front() = 'm';
10 std::cout << s << std::endl; // output: moor
```

Appending. For strings, the `+=` operator can be used for appends.

```
1 std::string s {"rail"};
2 s.pop_back(); // s: "rai"
3 s.push_back('n'); // s: "rain"
4 // can use += to append characters
5 s += 's'; // s: "rains"
6 // can use += to append strings
7 s += "torm" // s: "rainstorm"
```

⁸`std::string` is not a subclass of `std::vector<char>`. It just functions similarly, storing a contiguous character array (as of C++11), and has a similar interface.

⁹Technically, string literals are immutable in C++, but string variables are not immutable

Concatenation. Two strings can be concatenated (chained) using the + operator. Whenever possible, prefer using `s += t` instead of `s = s + t`, which is slower.

```
1 std::string s {"know"};
2 std::string t {"ledge"};
3 std::cout << s + t << std::endl; // output: knowledge
```

Substrings. The substring function in C++ works differently from languages like Java and Python! The `substr(pos, count)` function returns a `std::string` which is the substring starting at `pos`, and containing `count` characters.

```
1 std::string s {"hello world"};
2 std::cout << s.substr(6, 3) << std::endl; // output: wor
3 std::cout << s.substr(0, 2) << std::endl; // output: he
```

Omitting the second argument takes the suffix starting at index `pos`.

```
1 std::string s {"hello world"};
2 std::cout << s.substr(6) << std::endl; // output: world
3 std::cout << s.substr(8) << std::endl; // output: rld
```

Using <algorithm>. Similar to `std::vector`, use `begin()` and `end()` as arguments to functions in `<algorithm>`.

```
1 std::string s {"binarysearchtree"};
2 std::sort(s.begin(), s.end());
3 std::cout << s << std::endl; // output: aabceeehinrrsty
4
5 std::string t {"tACoCaT"};
6 std::reverse(t.begin(), t.end());
7 std::cout << t << std::endl; // output: TaCoCAt
```

Finding characters and substrings. The `find()` function returns the index of the first occurrence of the given substring or character. The `rfind()` function returns the index of the last occurrence of the given substring or character. For both functions, if the desired substring or character is not found, the special value `string::npos` is returned.

```
1 std::string s {"This is a string!"};
2 // technically unsigned int, but just using int is ok for comp prog
3 int i = s.find("is");
4 if (i != string::npos)
5 {
6     std::cout << i << std::endl;
7 }
8 else
9 {
10     std::cout << "not found" << std::endl;
11 }
```

Type conversion.¹⁰ Use `std::stoi` (string to int) to convert a string to an **int**, `std::stoll` to convert a string to a **long long**, and `std::to_string` to convert a numeric type to a string. These are all part of the `<string>` header.

```
1  int a {std::stoi("125")};
2  long long b {std::stoll("1250")};
3  std::string s {std::to_string(232)};
4  std::string t {std::to_string(232LL)};
```

¹⁰The functions in the `<cstdlib>`, which includes `atoi`, `strtoll`, and similar functions, are meant for C-style strings, which you shouldn't be using anyway.

2 Control structures

2.1 The basics you should know

Programming languages generally share common control structures, such as conditionals and loops. Read up on the following so you are familiar with the basic syntax for control structures in C++.

- [learncpp.com - 8.1 - Control flow introduction](#)
- [learncpp.com - 8.2 - If statements and blocks](#)
- [learncpp.com - 8.3 - Common if statement problems](#)
- [learncpp.com - 8.5 - Switch statement basics](#)
- [learncpp.com - 8.6 - Switch fallthrough and scoping](#)
- [learncpp.com - 8.8 - Introduction to loops and while statements](#)
- [learncpp.com - 8.9 - Do while statements](#)
- [learncpp.com - 8.10 - For statements](#)
- [learncpp.com - 8.11 - Break and continue](#)
- [learncpp.com - 16.8 - Range-based for loops \(for-each\)](#)

2.2 Some common patterns

Here, we'll show some examples of very common competitive programming tasks. These are basically meant for you to adapt into your own programming style.

2.2.1 Reading a list of n integers

A range for loop works with both vectors and C-style arrays.

```
1  #include <iostream>
2  #include <vector>
3  int main()
4  {
5      int n; std::cin >> n;
6      std::vector<int> V(n);
7      for (auto& x : V) std::cin >> x;
8  }
```

```
1  #include <iostream>
2  int main()
3  {
4      int n; std::cin >> n;
5      int A[n] {};
6      for (auto& x : A) std::cin >> x;
7  }
```

Remarks:

- We use **auto&** instead of just **auto** so that we obtain a *reference* to each vector/array element. That way, when we modify x (using `std::cin`), we also modify the respective vector/array element. **This is also more efficient, since we avoid making copies of the elements of the vector/array if we use `auto&` instead of `auto`.**
- Using a C-style array like this is less common, because of the limits on how big of an array you can declare on the stack.

Another common style is to declare a global array which is large enough to handle all input sizes. This may make it easier for you to have an idea of how much memory your program is using.

```
1  #include <iostream>
2  int A[200000];
3  int main()
4  {
5      int n; std::cin >> n;
6      for (int i = 0; i < n; i++) std::cin >> A[i];
7  }
```

2.2.2 Reading an $R \times C$ grid

An $R \times C$ grid has R rows and C columns. A common input format for a grid is something like this, where the first line indicates the number of rows and columns, and then R lines follow containing the rows.

Sample Input

```
3 4
0 1 2 3
4 5 6 7
8 9 10 11
```

You can think of a grid as a “list of rows”. Since a row of integers can be represented using a `std::vector<int>`, a list of rows is a `std::vector<std::vector<int>>`.

```
1  #include <iostream>
2  #include <vector>
3  int main() {
4      int R, C; std::cin >> R >> C;
5      std::vector<std::vector<int>> grid (R);    // there are R rows
6      for (auto& row : grid) row.resize(C);    // each row has C columns
7
8      for (auto& row : grid) {                // for each row
9          for (auto& el : row) {              // for each element in row
10             std::cin >> el;
11         }
12     }
13
14     // alternate method
15     for (int i = 0; i < R; i++) {
16         for (int j = 0; j < C; j++) {
17             std::cin >> grid[i][j];
18         }
19     }
20 }
```

For a grid of characters (not separated by spaces), you can represent each row with a string.

Sample Input

```
3 4
abcd
efgh
ijkl
```

```
1  #include <iostream>
2  #include <string>
3  #include <vector>
4  int main() {
5      int R, C; std::cin >> R >> C;
6      std::vector<std::string> grid (R);
7      for (auto& row : grid) std::cin >> row;
8  }
```

2.2.3 Passing vectors to functions

A common code organization style in competitive programming is to separate the logic which solves the problem from the logic which handles input and output. Pay attention to the following code.

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  // this line makes it so that we can type ll instead of "long long"
6  using ll = long long; // this is a better version of typedef
7
8  ll solve(vector<ll>& V)
9  {
10     ll ans {};
11     // ... do things ...
12     return ans;
13 }
14
15 int main()
16 {
17     int n; cin >> n;
18     vector<ll> V(n);
19     for (auto& x : V) cin >> x;
20
21     cout << solve(V) << endl;
22 }
```

Notice that the `solve` function takes an argument which is a **reference** to a vector. In other words, note that we wrote

```
long long solve(std::vector<long long>& V)
```

and not

```
long long solve(std::vector<long long> V)
```

The first style is called **pass-by-reference**, and the second style is called **pass-by-value**. The short explanation is:

- Pass-by-value makes a copy of the argument, while pass-by-reference passes a reference to the original object. This avoids making copies and is therefore more efficient when passing large amounts of data.
- You would only want to pass by reference "large" types, like `vector` and `string`. Passing primitive types, like `int`, `char` or `long long`, by reference has negligible effect on performance.

In competitive programming, while global variables are often used to share data across multiple functions, a more organized way to do so is by passing references.

For the long explanation, consider the following example.

```
1  #include <iostream>
2
3  // A "void" function is a function which does not return a value.
4  void pass_val(int x)
5  {
6      x += 2;
7      std::cout << "pass_val(): " << x << std::endl;
8  }
9
10 void pass_ref(int& x)
11 {
12     x += 2;
13     std::cout << "pass_ref(): " << x << std::endl;
14 }
15
16 int main()
17 {
18     int x {5};
19     pass_val(x);
20     std::cout << "main(): " << x << std::endl;
21     pass_ref(x);
22     std::cout << "main(): " << x << std::endl;
23 }
```

Sample Output

```
pass_val():7
main():5
pass_ref():7
main():7
```

- The execution of the program starts at `main()`. Since the parameter of `pass_val` is `int` and not `int&`, in order to execute `pass_val`, we make a copy of `x`, and have `pass_val` operate on the copy.
- Since the copy held by `pass_val` is separate from the `x` held locally in `main`, the second line of output says 5. The increment done in `pass_val` does not affect the `x` in `main`.
- Since the parameter of `pass_ref` is `int&` and not `int`, in order to execute `pass_ref`, we make a reference bound to `x`, and have `pass_ref` operate on the reference.
The `x` in `pass_ref` and the `x` in `main` now refer to the same variable.

3 Pairs, tuples, and structs

3.1 `std::pair`

There are many problems where it is natural to bundle together related data. The `<utility>` header allows you to create **pairs**. A `std::pair` contains two values (of any type), and a `std::tuple` (introduced in C++11) is a generalization of pair which allows you to store a bundle of any number of values, all of different types.

To declare a `std::pair`, we have to include the types of each element in angle brackets.

```
1 std::pair<int, int> P; // P is a pair where the first and second elements are ints
2 std::pair<double, long long> Q; // first element: double, second element: long long.
```

When declaring a pair, we can indicate its elements in parentheses or braces. To create a pair value, we can use the helper function `std::make_pair()`. The return type of `std::make_pair` is automatically determined based on the type of the arguments.

```
1 #include <utility>
2
3 int main()
4 {
5     std::pair<int, int> p1 (1, 2);
6     std::pair<int, bool> p2 {0, true};
7     auto p3 = std::make_pair(3, 4);
8 }
```

To access the elements of a pair, we use the `first` and `second` properties.

```
1 #include <iostream>
2 #include <utility>
3
4 int main()
5 {
6     std::pair<int, int> p1 (1, 2);
7
8     std::cout << p1.first << " " << p1.second << std::endl; // output: 1 2
9
10    p1.first = 3 + p1.second;
11
12    std::cout << p1.first << " " << p1.second << std::endl; // output: 5 2
13 }
```

To demonstrate various ways of working with pairs, suppose we have a problem where the input specification consists of n pairs of integers. So the first line of input contains a positive integer n , and this is followed by n lines, each containing two space-separated integers.

Sample Input

```
5
1 -2
4 2
5 -4
2 -6
7 -1
```

A natural choice of container for this data would be a `std::vector<std::pair<int, int>>`. We could populate this container with repeated use of `push_back`, like this:

```
1 std::vector<std::pair<int, int>> data;
2 int n; std::cin >> n;
3 for (int i = 0; i < n; i++) {
4     int x, y; cin >> x >> y;
5     data.push_back(std::make_pair(x, y));
6 }
```

Another approach is use the `resize` function of a vector to ensure it has exactly n pairs, and then read in the data directly into each pair.

```
1 std::vector<std::pair<int, int>> data;
2 int n; std::cin >> n;
3 data.resize(n);
4 for (std::pair<int, int>& p : data) { // can also use auto&
5     std::cin >> p.first >> p.second;
6 }
```

Notice how we use `first` and `second` to access the elements of a pair in the above loop. Since C++17, a feature called **structured binding**¹¹ allows us to bind variable names to elements of a pair or tuple. It's easier to understand with an example.

```
1 std::vector<std::pair<int, int>> data;
2 int n; std::cin >> n;
3 data.resize(n);
4 for (auto& [x, y] : data) { // you must use auto& here
5     std::cin >> x >> y;
6 }
```

The list of variable names is given in brackets, in either the form¹² `auto [ <variable names> ]` or `auto& [ <variable names> ]`. The variable names are separated by commas.

In the above example, the for-each loop goes through each `std::pair<int, int>` found in the vector `data`. And for each pair, the first element of the pair is bound (by reference) to the variable name `x`, and the second element of the pair is bound (by reference) to the variable name `y`.

Structured binding can be used to unpack a pair in general.

```
1 std::pair<int, int> p (1, 2);
2
3 // a and b are both int, where a = p.first and b = p.second
4 auto [a, b] = p;
5
6 // r1 and r2 are both int&, where r1 = p.first and r2 = p.second
7 auto& [r1, r2] = p;
```

¹¹Documentation available at https://en.cppreference.com/w/cpp/language/structured_binding.

¹²There are other forms we won't discuss, see the documentation.

Sometimes, when solving a problem, you want to manipulate a `std::vector<std::pair<int, int>>` and you want to do something which requires the indices of the elements.

```
1 std::vector<std::pair<int, int>> data;
2 // ... fill up data ...
3 for (int i = 0; i < data.size(); i++)
4 {
5     // Do something with data[i].first
6     // Do something with data[i].second
7 }
```

One way to make your code easier to read and write would be to declare references inside the loop, so that you now have convenient variable names for `pair.first` and `pair.second`.

```
1 std::vector<std::pair<int, int>> data;
2 // ... fill up data ...
3 for (int i = 0; i < data.size(); i++)
4 {
5     int& x = data[i].first;    // can also do auto&
6     int& y = data[i].second;  // can also do auto&
7
8     // Now you can work with x and y, while also knowing the index i
9     // Do something with x
10    // Do something with y
11 }
```

Using structured binding, we can also write

```
1 std::vector<std::pair<int, int>> data;
2 // ... fill up data ...
3 for (int i = 0; i < data.size(); i++)
4 {
5     auto& [x, y] = data[i];
6
7     // Now you can work with x and y, while also knowing the index i
8     // Do something with x
9     // Do something with y
10 }
```

3.2 std::tuple

Tuples, defined in the `<tuple>` header, are similar to pairs, but you can have more than two elements. We use the `std::make_tuple` function to create tuples, and we use the `std::get<>` function to obtain elements of a tuple.

```
1  #include <iostream>
2  #include <string>
3  #include <tuple>
4
5  int main()
6  {
7      std::tuple<int, int, std::string> t (420, 68, "lazy");
8      auto s = std::make_tuple(11,22,33,44);
9      std::cout << std::get<0>(t) << std::endl;    // output: 420
10     std::cout << std::get<3>(s) << std::endl;    // output: 44
11
12     std::get<2>(t) = "sleepy";
13     auto [a,b,c] = t;
14     std::cout << b << " " << c << std::endl;    // output: 68 sleepy
15 }
```

And to read/manipulate input that looks like this, structured binding can be used with for loops involving tuples, just like pairs.

Sample Input

```
5
1 -2 3
4 2 4
5 -4 5
2 -6 6
7 -1 7
```

```
1  #include <iostream>
2  #include <vector>
3  #include <tuple>
4
5  int main()
6  {
7      std::vector<std::tuple<int,int,int>> data;
8      int n; std::cin >> n;
9      data.resize(n);
10     for (auto& [x,y,z] : data)
11     {
12         std::cin >> x >> y >> z;
13     }
14     for (int i = 0; i < data.size(); i++)
15     {
16         auto& [x,y,z] = data[i]
17         // do something to x,y,z, while also knowing i
18     }
19 }
```

Those of you used to coding in Python are probably happy right now that you can write code that looks like the above in C++.

3.3 Structs

I'm working on it.

3.4 Custom sorting

I'm working on it.

3.5 A couple notes on list-initialization

When creating an object in C++, you generally have to indicate the type of the object you are creating.

```
1  #include <vector>
2  #include <utility>
3  #include <tuple>
4
5  struct item
6  {
7      int weight;
8      int value;
9  };
10
11 int main()
12 {
13     std::vector<int> v {1,2,3,4};
14     std::pair<int, int> p {2, 4};
15     std::tuple<int, int, int> t {1,2,3};
16
17     item I {4, 5};
18 }
```

When using **braces**, C++ can even handle nested structures. Check this out!

```
1  #include <vector>
2  #include <utility>
3
4  struct item
5  {
6      int weight;
7      int value;
8  };
9
10 int main()
11 {
12     std::vector<std::vector<int>> v {{1,2,3,4}, {5,6,7,8}, {9,10,11,12}};
13     std::pair<item, int> p {{1,2}, 5};
14 }
```

And, if the type of the object can be inferred from context, then we don't even need to specify the type!

For example, if you have a `std::vector<std::pair<int,int>> V`, then `V.push_back` *expects* a `std::pair<int,int>`. Since the type of the expected argument to `V.push_back` is obvious, we don't need to actually write it!

```
1  #include <vector>
2  #include <utility>
3
4  struct item
5  {
6      int weight;
7      int value;
8  };
9
10 int main()
11 {
12     std::vector<std::pair<int,int>> V;
13     V.push_back({1,2});
14     V.push_back({4,5});
15     std::vector<std::pair<item,int>> W;
16     W.push_back({{1,2}, 3});
17     W.push_back({{1,3}, 6});
18 }
```

Similarly, if the *return type* can be initialized using braces, like a vector, pair, tuple, or struct, we don't need to specify the type in the return statement. The following function returns a vector containing 0, 1, or 2 doubles.

```
1  #include <vector>
2  #include <cmath>
3
4  // solves the quadratic  $ax^2 + bx + c = 0$ 
5  std::vector<double> solve(int a, int b, int c)
6  {
7      int discriminant = b*b - 4*a*c;
8
9      if (discriminant < 0) return {};
10
11     if (discriminant == 0) return {-b / (double)(2*a)};
12
13     return {
14         (-b + sqrt(discriminant)) / (double)(2*a),
15         (-b - sqrt(discriminant)) / (double)(2*a)
16     };
17 }
```